Conoce el x500

Del dron real a tu primer nodo de ROS 2 (Parte 1)

Imad Jared Cabrera Trejo · Vespo UAV · 11 de noviembre de 2026



Hoy

1. **El dron**: qué piezas tiene y qué hace cada una.
2. **Su gemelo**: el mismo x500, en la simulación.
3. **Quién hace qué**: tu programa, PX4, QGroundControl y Gazebo.
4. **El viaje de un comando**: qué pasa desde que das Enter hasta que el dron sube.
5. **Lo que vas a ver** en tu pantalla.
6. **Tu primer vuelo**, paso a paso.



Foto: documentación de PX4 (CC BY 4.0)

Contenido

1. El dron
2. Su gemelo en la simulación
3. Quién hace qué
4. El viaje de un comando
5. Lo que vas a ver
6. Ideas para el código
7. Tu primer vuelo

Este es el x500



Holybro X500 V2. Foto: documentación de PX4 (CC BY 4.0)

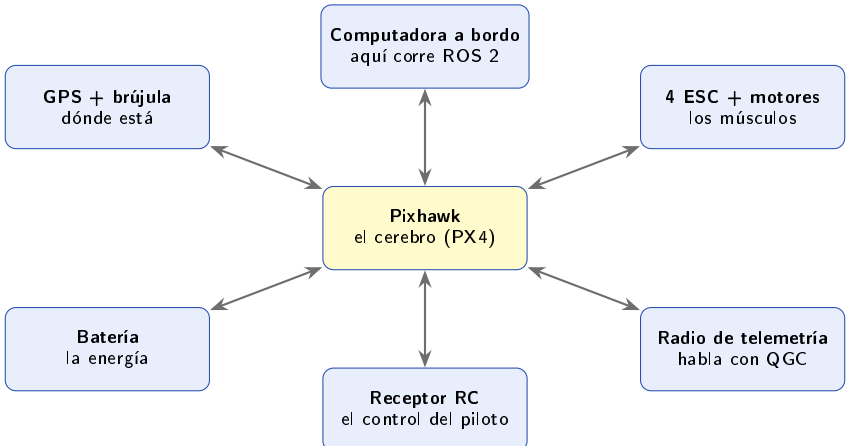
- Un **cuadricóptero**: 4 motores, marco en X.
- ~50 cm de motor a motor (en diagonal).
- Marco de fibra de carbono, hélices de 10 pulgadas.
- Es el dron que usa PX4 **por defecto** en simulación: lo que aprendan aquí sirve para el real.

¿Qué trae adentro?



El kit del X500 desarmado, cada pieza con su nombre. Foto: documentación de PX4 (CC BY 4.0)

Las piezas, de un vistazo



El cerebro: la Pixhawk 6C



Foto: documentación de PX4 (CC BY 4.0)

- Una computadora pequeña dedicada a **volar**. Ahí corre **PX4**.
- Trae adentro la **IMU** (giroscopio y acelerómetro) y el **barómetro**.
- Decide cuánto girar cada motor, **cientos de veces por segundo**.
- Nosotros no la programamos por dentro: le **pedimos** cosas desde ROS 2.

En ROS 2 la IMU aparece como `/fmu/out/sensor_combined` y lo que PX4 sabe del dron como `/fmu/out/vehicle_status_v1`.

Tiene un frente

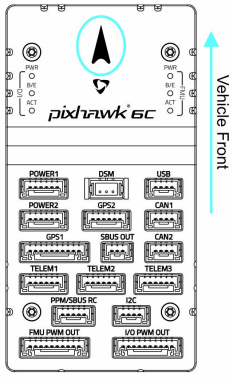


Foto: documentación de PX4 (CC BY 4.0)

- La flecha de la Pixhawk tiene que apuntar al **frente del dron**.
- PX4 mide todo respecto a ella: si la montas al revés, cree que avanza cuando retrocede.
- En PX4 ese marco se llama **FRD**: x al **F**rente, y a la de**R**echa, z hacia a**D**entro de la tierra (abajo).

Dónde está: GPS y brújula



Holybro M9N. Foto: documentación de PX4 (CC BY 4.0)

- El **GPS** da latitud y longitud (unos 5–10 veces por segundo, con ~1 m de error).
- Trae una **brújula** (magnetómetro): sabe hacia dónde está el norte.
- Va arriba, en un mástil, lejos de los cables (los motores hacen "ruido" magnético).

En ROS 2: el GPS crudo en `/fmu/out/vehicle_gps_position`, y la posición que PX4 calcula combinando todo en `/fmu/out/vehicle_global_position`.

Los músculos: motores, ESC y hélices



Foto: documentación de PX4 (CC BY 4.0)

- Cada brazo lleva un **motor** sin escobillas con su **hélice**.
- Cada motor tiene un **ESC**: el "traductor" entre la orden de la Pixhawk ("75 %") y la corriente que mueve el motor.
- Dos giran a un lado y dos al otro (luego vemos por qué).

Ojo

Las hélices cortan. Toda prueba de motores en el dron real va **sin hélices**.

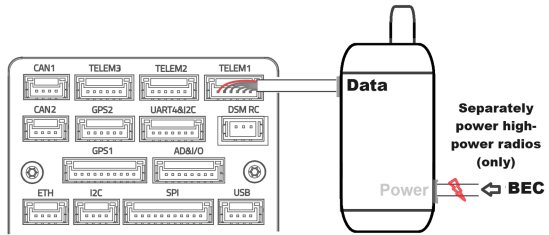
Energía y comunicación

Batería LiPo (4S)

- 4 celdas de ~4.2 V llenas: ~16.8 V.
- Un vuelo dura minutos: hay que vigilarla.
- En ROS 2: `/fmu/out/battery_status_v1`.

Control remoto (RC)

- El piloto de seguridad: puede tomar el control en cualquier momento.



Radio de telemetría conectada a la Pixhawk. Documentación de PX4 (CC BY 4.0)

La **radio de telemetría** conecta el dron con **QGroundControl** en tu laptop.

De la pieza al topic

En el dron	Lo que mide o hace	En ROS 2 lo lees en
IMU (en la Pixhawk)	giros y aceleraciones	`/fmu/out/sensor_combined`
GPS	latitud, longitud	`/fmu/out/vehicle_gps_position`
PX4 combinando todo	dónde está, en metros	`/fmu/out/vehicle_local_position_v1`
PX4 combinando todo	hacia dónde apunta	`/fmu/out/vehicle_attitude`
Batería	voltaje, %	`/fmu/out/battery_status_v1`
PX4 (el piloto)	armado, modo de vuelo	`/fmu/out/vehicle_status_v1`
Motores (vía PX4)	a dónde ir	`/fmu/in/trajectory_setpoint`
PX4 (el piloto)	órdenes: armar, aterrizar...	`/fmu/in/vehicle_command`

`out` = PX4 nos cuenta. `in` = le pedimos algo.

Contenido

1. El dron
2. Su gemelo en la simulación
3. Quién hace qué
4. El viaje de un comando
5. Lo que vas a ver
6. Ideas para el código
7. Tu primer vuelo

El mismo dron, en la computadora





En Gazebo

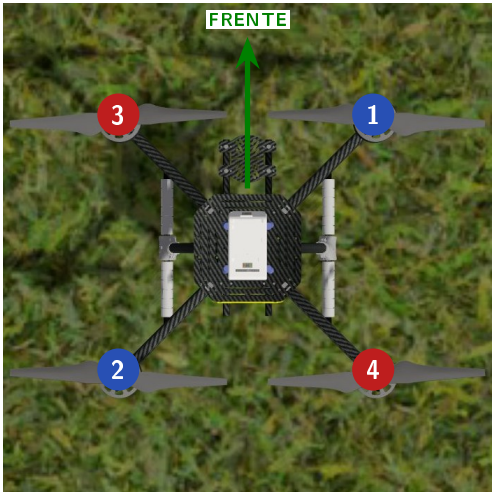
Render propio de Gazebo

Visto de cerca



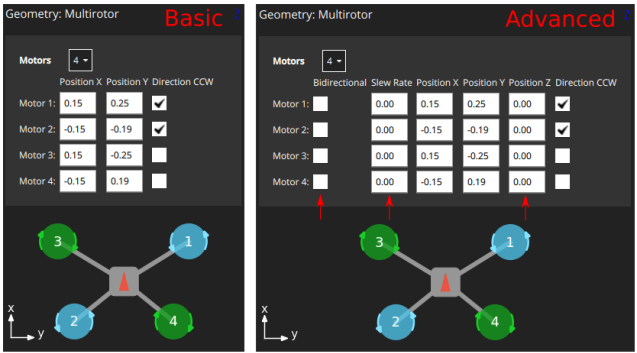
El x500 de la simulación de PX4. Render propio de Gazebo

Los motores, desde arriba



- 1 y 2 giran **antihorario**.
- 3 y 4 giran **horario**.
- Los que giran igual van en diagonal: así sus "torceduras" se cancelan y el dron no da vueltas solo.
- Para **girar**, PX4 acelera un par y frena el otro.

Lo mismo en QGroundControl



Pantalla de motores de QGroundControl. Documentación de PX4 (CC BY 4.0)

- QGC dibuja el mismo orden: el triángulo rojo es el frente.
- Ahí se prueba cada motor en el dron real (o con `motor_test` desde ROS 2).
- Si dos motores están cruzados, **el dron se voltea al despegar**.

Con sensores extra



x500_lidar_2d: un lidar arriba que mide distancias a 360° (como un radar de luz).



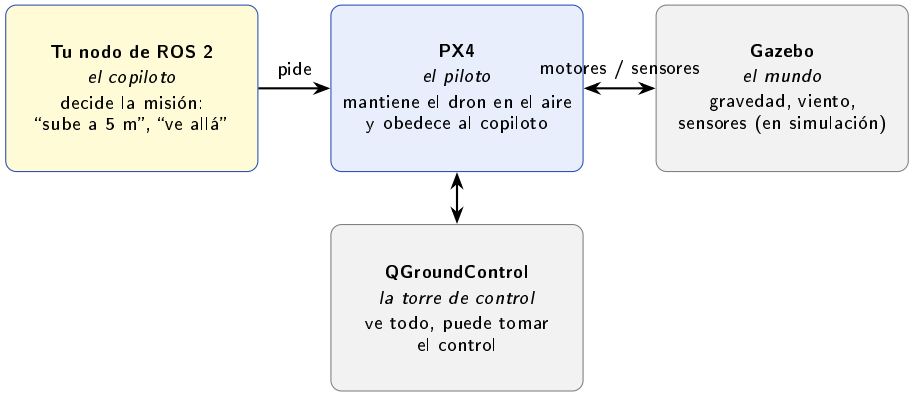
x500_cam_abajo: una cámara que mira hacia el suelo, para buscar marcadores.

Renders propios de Gazebo

Contenido

1. El dron
2. Su gemelo en la simulación
3. Quién hace qué
4. El viaje de un comando
5. Lo que vas a ver
6. Ideas para el código
7. Tu primer vuelo

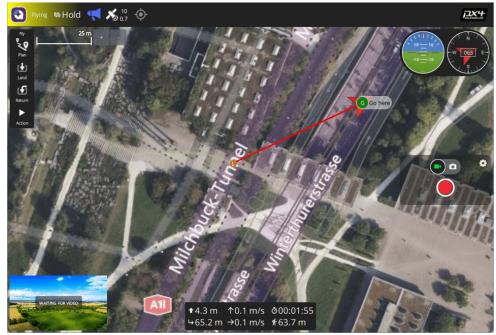
Cuatro programas, cuatro papeles



El piloto y el copiloto

Piensen en un avión:

- El **piloto** (PX4) tiene las manos en los controles: mantiene el equilibrio, corrige el viento, nunca suelta.
- El **copiloto** (tu nodo) lleva el mapa y le dice a dónde ir: “sube a 5 m”, “ahora al norte”.
- Si el copiloto se desmaya (tu nodo se cae), el piloto **no se estrella**: hace lo seguro (quedarse quieto, regresar o aterrizar).



QGroundControl, vista de vuelo. Documentación de PX4 (CC BY 4.0)

Dos formas de darle instrucciones al piloto

“Sigue mi mano” (offboard)

El copiloto dice **a cada instante** dónde estar: 10–20 veces por segundo.

- Para trayectorias propias: un círculo, un 8, seguir algo.
- Si deja de hablar medio segundo, el piloto toma el control.

“Ve a tal lugar” (comandos)

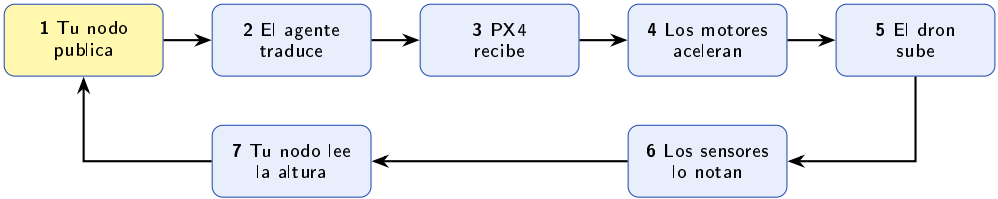
El copiloto da una **orden** y espera: “despega”, “ve a este punto GPS”, “regresa”.

- El piloto vuela con sus propias maniobras.
- Si el copiloto se calla, el piloto se queda quieto.

Contenido

1. El dron
2. Su gemelo en la simulación
3. Quién hace qué
4. El viaje de un comando
5. Lo que vas a ver
6. Ideas para el código
7. Tu primer vuelo

El viaje de un comando: “sube a 5 metros”

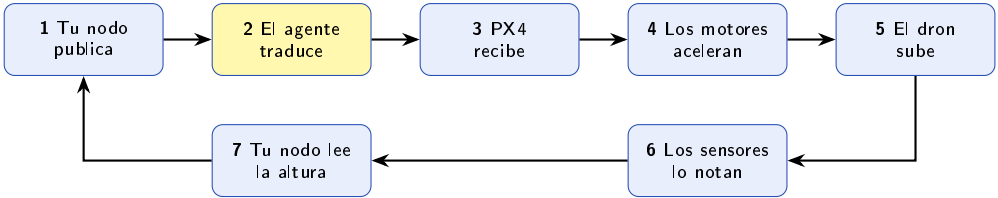


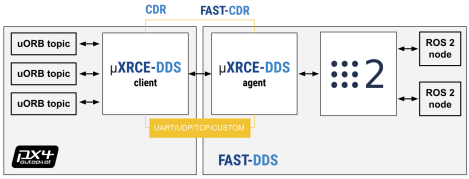

```
ros2 run px4_examples takeoff_mission
```

```
Comandos OFFBOARD + ARM enviados
alt=0.04 m
```

Corres el programa. 10 veces por segundo publica un **setpoint**: “quiero estar en $(0, 0, -5)$”. Al segundo pide **modo offboard** y **armar**.

El viaje de un comando: “sube a 5 metros”

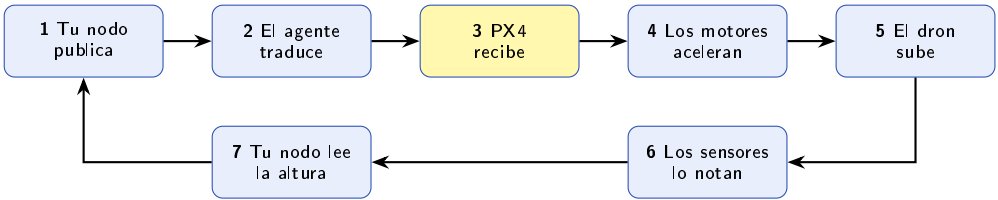




Documentación de PX4 (CC BY 4.0)

El **agente** (MicroXRCEAgent) toma los mensajes de ROS 2 y se los pasa a PX4 por UDP (en el dron real, por un cable serial).

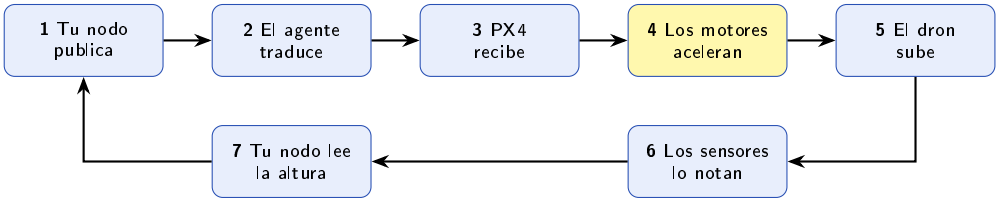
El viaje de un comando: “sube a 5 metros”





PX4 revisa que todo esté bien (sensores, GPS, batería), **arma** y entra a **offboard**: desde ahora obedece tus setpoints.

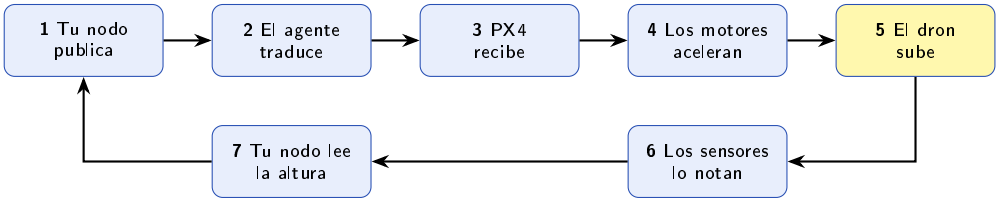
El viaje de un comando: “sube a 5 metros”





Los controladores de PX4 calculan cuánto empuje necesita cada motor para subir sin inclinarse, **cientos de veces por segundo**.

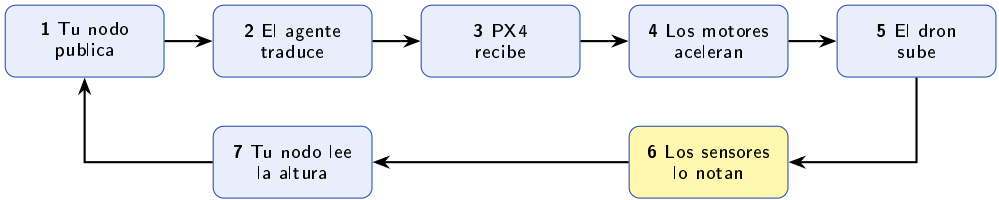
El viaje de un comando: “sube a 5 metros”





En la simulación, **Gazebo** aplica ese empuje, la gravedad y el viento: el dron sube. En el real, lo hace la física.

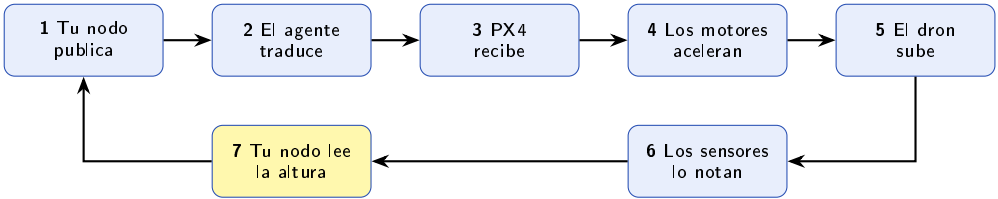
El viaje de un comando: “sube a 5 metros”





La IMU, el barómetro y el GPS miden el movimiento. El **EKF2** de PX4 los combina y calcula “estoy a 4.98 m”.

El viaje de un comando: “sube a 5 metros”

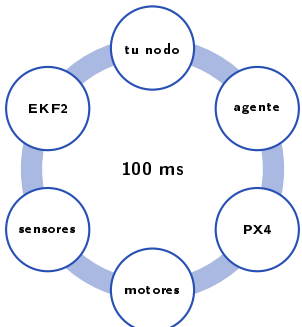


takeoff_mission

```
alt=4.98 m
Arriba: mantengo 5 m
```

PX4 publica esa estimación en `vehicle_local_position_v1`; tu nodo la lee y decide qué sigue. **Y el ciclo empieza otra vez.**

Todo eso pasa 10 veces por segundo



- Tu nodo da una vuelta cada **100 ms** (un *timer*).
- Por dentro, PX4 da **muchas más** vueltas: sus motores se ajustan cientos de veces entre uno de tus setpoints y el siguiente.
- Por eso tú solo dices **a dónde**, no cómo mover cada motor.

¿Y si tu nodo se cae?

En offboard, PX4 espera tus setpoints **todo el tiempo**. Es como el **pedal del hombre muerto** de los trenes: si el maquinista lo suelta, el tren frena solo.

- Si dejan de llegar setpoints (~0.5 s), PX4 entra en **failsafe**: se queda quieto, regresa o aterriza.
- Por eso el nodo publica aunque "no cambie nada".

flight_mode_monitor (al dar Ctrl+C)

```
Modo: Offboard -> Hold
FAILSAFE ACTIVO (modo Hold)
Banderas de failsafe:
señal offboard perdida
```

Contenido

1. El dron
2. Su gemelo en la simulación
3. Quién hace qué
4. El viaje de un comando
5. Lo que vas a ver
6. Ideas para el código
7. Tu primer vuelo

Un comando levanta todo

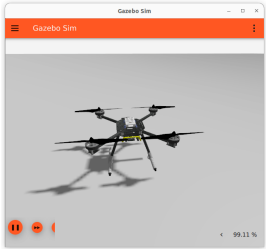
```
vespo up --sim
```

Terminal: agente
MicroXRCEAgent
(el traductor)

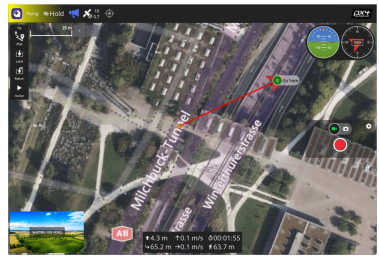
Terminal: PX4
la consola `pxh>`
(mensajes de PX4)

Ventana: Gazebo
el mundo y el dron

Ventana: QGC
mapa, modo, altura

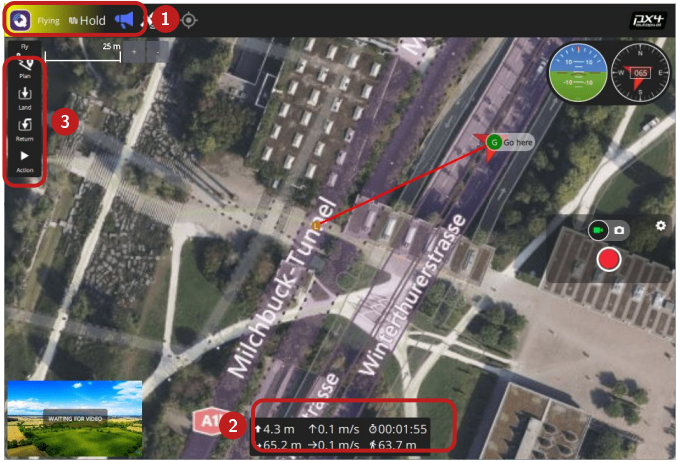


Gazebo. Documentación de PX4 (CC BY 4.0)



QGroundControl. Documentación de PX4 (CC BY 4.0)

En QGroundControl, mira estas tres cosas



1. **Estado y modo**: “Flying”, “Hold”, “Offboard”... Lo primero que hay que ver cuando algo no jala.
2. **Altura y velocidad**: la telemetría en vivo.
3. **Botones de seguridad**: Land, Return... el piloto siempre puede tomar el control.

Documentación de PX4 (CC BY 4.0)

En la terminal

ros2 run px4_examples telemetry_listener

```
Estado: desarmado | modo: HOLD (4)
Local: x=0.01 y=-0.02 alt=0.03 m
GPS: lat=47.3979711 lon=8.5461637
Actitud: roll=0.1° pitch=-0.2° yaw=90.0°
Batería: 16.20 V | 100 %
```

Si sale **"Sin datos de vehicle_status todavía"**: el agente o PX4 no están corriendo.

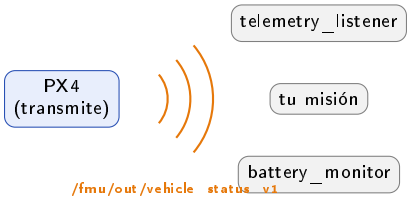
ros2 topic list | grep fmu

```
/fmu/in/offboard_control_mode
/fmu/in/trajectory_setpoint
/fmu/in/vehicle_command
/fmu/out/battery_status_v1
/fmu/out/vehicle_local_position_v1
/fmu/out/vehicle_status_v1
...
```

Contenido

1. El dron
2. Su gemelo en la simulación
3. Quién hace qué
4. El viaje de un comando
5. Lo que vas a ver
6. Ideas para el código
7. Tu primer vuelo

Un topic es una estación de radio



- El **publisher** transmite en una frecuencia con nombre (`/fmu/out/vehicle_status_v1`).
- Cualquier **subscriber** que sintonice ese nombre escucha. Pueden ser muchos.
- El que transmite no sabe quién escucha, ni le importa.

Radio en vivo o carta certificada (QoS)

best_effort: radio en vivo

Si te perdiste un mensaje, ya pasó; el siguiente llega en milisegundos. Así publica PX4: a 50 Hz no vale la pena repetir datos viejos.

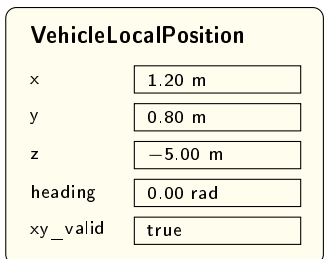
reliable: carta certificada

Se reenvía hasta confirmar que llegó. Lo usamos entre nodos nuestros (`/vespo/...`).

Ojo

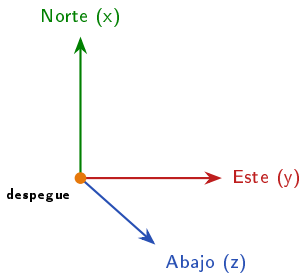
Si pides carta certificada a una estación de radio, **no te llega nada**. Por eso los ejemplos se suscriben a PX4 con `best_effort`.

Un mensaje es un formulario



- Cada tipo de mensaje tiene sus **casillas** (campos) con nombre y tipo.
- PX4 llena el formulario y lo publica; tu nodo lo lee: `position_.z`.
- Para ver las casillas de un tipo:
  `ros2 interface show px4_msgs/msg/VehicleLocalPosition`

NED: como un submarino



PX4 mide como un submarino: “cuántos metros al norte, al este y **de profundidad**”.

- $z = 0$: en el suelo donde arrancó.
- $z = -5$: **5 m arriba** (profundidad negativa).
- Por eso en los ejemplos la altura se calcula como -z.

Callbacks y timers: el timbre y la alarma

Callback = el timbre

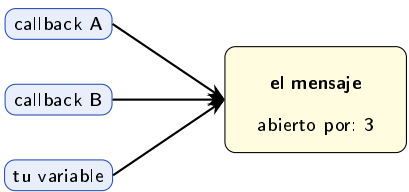
No te quedas en la puerta esperando el paquete: cuando llega, **suena el timbre** y ROS llama a tu función. En los ejemplos solo guarda el mensaje: `status_ = *msg`.

Timer = la alarma

Suena cada 100 ms y llama a `loop()`: ahí se decide qué hacer con lo último que llegó y se publica el setpoint.

Ninguno de los dos debe tardar: mientras uno corre, los demás esperan (un `sleep` adentro congela todo el nodo).

SharedPtr: un documento compartido



- Como un documento en la nube que varios tienen abierto: no se copia, todos ven el mismo.
- Lleva la cuenta de cuántos lo tienen abierto.
- Cuando el último lo cierra, **se borra solo**.
- Por eso en ROS 2 todo es `SharedPtr`: nadie tiene que acordarse de borrar nada.

Contenido

1. El dron
2. Su gemelo en la simulación
3. Quién hace qué
4. El viaje de un comando
5. Lo que vas a ver
6. Ideas para el código
7. Tu primer vuelo

Paso 1: compilar y levantar

```
cd /data/vespo_ws
vespo build                  # compila todo (la primera vez tarda)
vespo up --sim               # agente + PX4 + Gazebo + QGroundControl
```

Espera a que:

- Gazebo muestre el dron en el suelo,
- QGC diga **"Ready To Fly"** (puede tardar ~30 s mientras el GPS simulado y el EKF2 se acomodan).



Paso 2: escuchar

```
ros2 run px4_examples telemetry_listener     # en otra terminal
ros2 run px4_examples flight_mode_monitor    # en otra más
```

Déjenlos corriendo: les van a contar todo lo que pase en los siguientes pasos.

Si `telemetry_listener` no recibe nada, **no sigan**: primero hay que arreglar la conexión (¿corre el agente?, ¿corre PX4?).

Paso 3: despegar

```
ros2 run px4_examples takeoff_mission
```

Lo que debería pasar:

1. En `flight_mode_monitor`: **ARMADO** y **Hold** → **Offboard**.
2. En QGC el modo dice Offboard y la altura sube.
3. En Gazebo el dron se eleva y se queda a 5 m.



Paso 4: romperlo a propósito

Con el dron arriba, den **Ctrl+C** en `takeoff_mission`:

- El nodo deja de publicar setpoints.
- En ~0.5 s PX4 se da cuenta y entra en **failsafe**.
- Lo verán en `flight_mode_monitor` y en QGC.

Después aterricen con el botón **Land** de QGC.

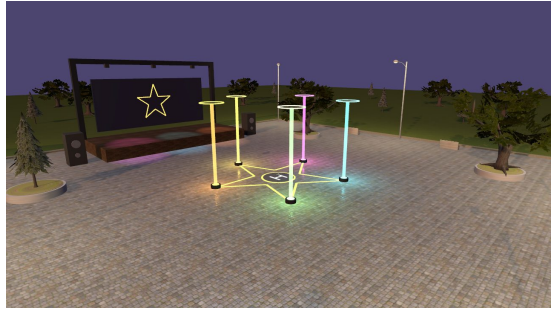
Así se aprende que tu nodo **no está solo**: PX4 siempre está vigilando, y cuando algo falla toma el control.

Y después

Parte 2: PX4 + ROS 2 con px4_examples

- Instalar todo con `vespo`, paso a paso.
- Escuchar al dron y volarlo: `square_mission` cuadro por cuadro.
- `shared_ptr`, comandos de PX4 y tu primer reto: la estrella.

Luego: las misiones de x500_retos, cada una en su propio mundo.



plaza_show, de x500_retos (render propio)

Créditos de las imágenes



- **Fotos del X500, la Pixhawk 6C, el GPS M9N, la radio de telemetría, QGroundControl, Gazebo y el diagrama de uXRCE-DDS**: documentación de PX4 (`docs.px4.io`), © PX4 / Dronecode y colaboradores, licencia **CC BY 4.0** (`creativecommons.org/licenses/by/4.0`). Solo se redimensionaron.
- **Renders del x500 y de los mundos**: propios, hechos con Gazebo Harmonic sobre el modelo `x500` de PX4 (`tools/renders_x500.py`).

